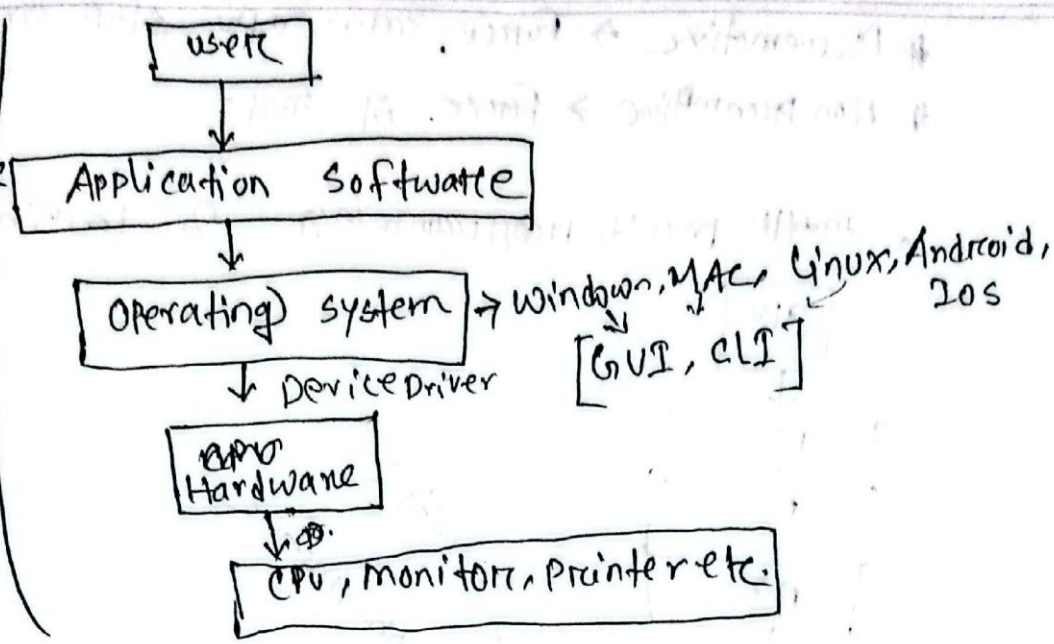
Operating System

25-01-26

* Operating system is a program that act as an interface between the user and the hardware. It is a resource allocator & Provides a Platform for the application Program.



Goal of Operating System

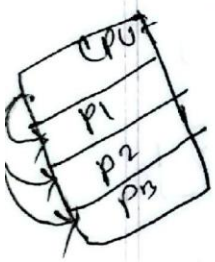
(i) Execute user Program and make solving user problem easier.

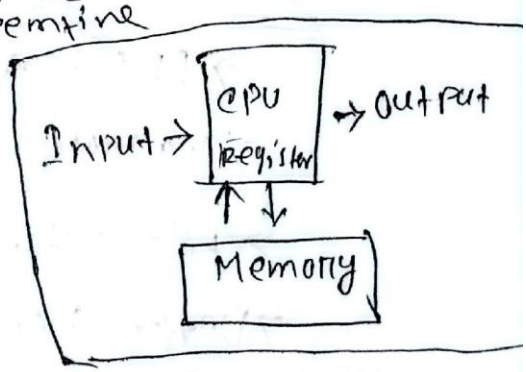
(ii) Make the computer system convenient to use.

(iii) use the computer hardware in an efficient manner.

(iv) Portability, (v) scalability (vi) Robustness

Types :- → Batch operating system
→ Multiprogramming → Non preemtive
→ Multi-tasking → preemtive
→ Multi-processing → mutiple CPU





* Inactive → program
* Active → process.

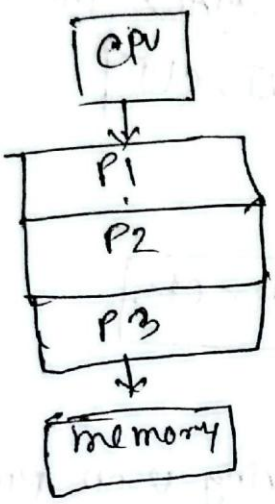
* HD → Secondary
* RAM → Primary | main
* ROM → primary | main
* Cache →
* Register →

CamScanner CS

Preemptive → Force করে CPU stop করায়।

Non-Preemptive → Force না করায়।

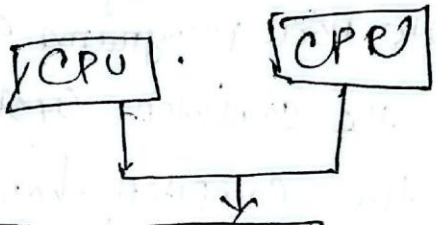
~~multi~~ multi programming & tasking.



Multiprocessing.

symmetric

Asymmetric



Operating system → memory Allocation / Deallocation

Service / Function

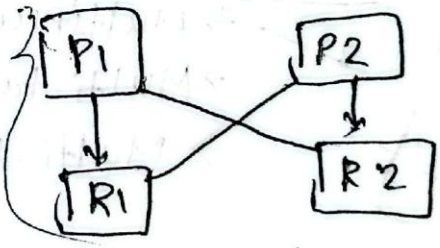
→ Process Management

→ Memory Management

→ File system Management

→ Device Management

→ etc.



Deadlock

Feb. 11 - 26

User input mode → 1 → user mode
Kernel mode → 0 → kernel mode

→ OS এর main অংশ হল kernel

system call

→ system /privilaged mode

[HW]

system call is a mechanism that allows a user-level process to request services from the operating system kernel

* scanf("%d", &a); → read করে, user থেকে kernel এ যায়
printf("%d", a); → write করে, kernel থেকে user এ switch করে।

* system call sequence to copy the contents of one file to another file.

Source file → Destination file

1. Acquire Input file name.
   - Write prompt to screen.
   - Accept input.
2. Acquire output file name.
   - Write prompt to screen
   - Accept Input.
3. Open the input file
   if file doesn't exist, abort
4. Create the output file.
   if file already exist, abort.
5. Read from input file.
6. Write to output file
7. Until read fails.
8. close output file.

(loop) continuously বার বার byte byte করে।

P.T.O

9. write completion message to screen.

10. Terminate normally.

Type of system call

- **Process control:**
  - → create & terminate process
  - → end, abort
  - → load, execute
  - → wait
  - → get process attribute, set process attribu
  - → allocate & free memory
- **File management:-**
  - → create, delete
  - → open, close
  - → read, write,
  - → get & set file attribute
- **Device Management:**
  - → request device
  - → Read, write
  - → get & set device attribute
- **Information maintenance:-**
  - → get time or date
  - → set time or date
  - → get & set process, file & device attribute.

CamScanner CS

02-02-26

~~# Type of~~ s

*why platform dependent? Application সুলো কি সব OS এ run করলে কাজ করবে? অন্য OS কী কাজ করবে?

windows
create process()
Exit process()

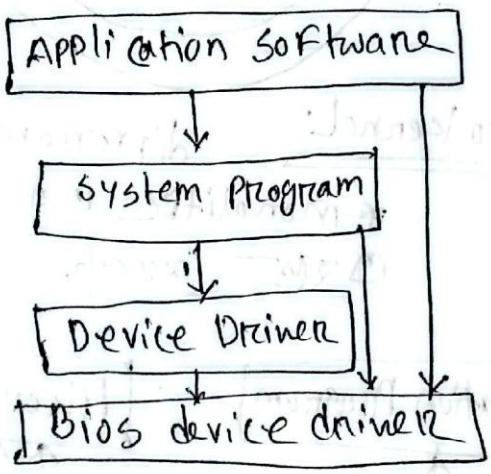
Linux
fork()
exit()

→ ~~open~~ application software are operating system specific.

Structure of OS

→ Simple structure: MS DOS

*Application software directly connected with OS. if one program crash whole system will be crashed.



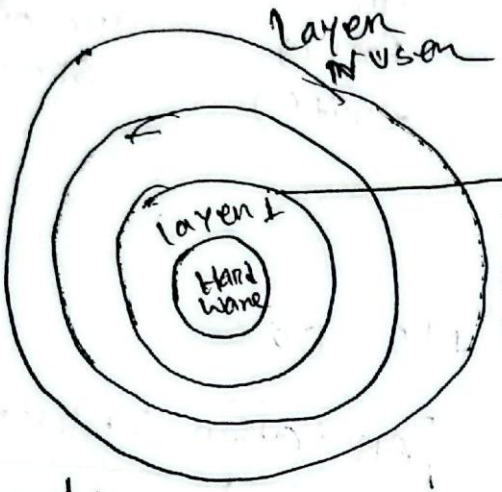
→ Monolithic: unix

	User	
shells & command, compiler & interpreter, system libraries		
~~Singn~~ Signal handling, terminal	filesystem	CPU scheduling, Page Replacement
Terminal controllers	Device controller	memory controller

Kernel

* Kernel এ কোনো Program crash হয়লে OS এ Problem হবে user এ কিছু হবে না।

> layered:

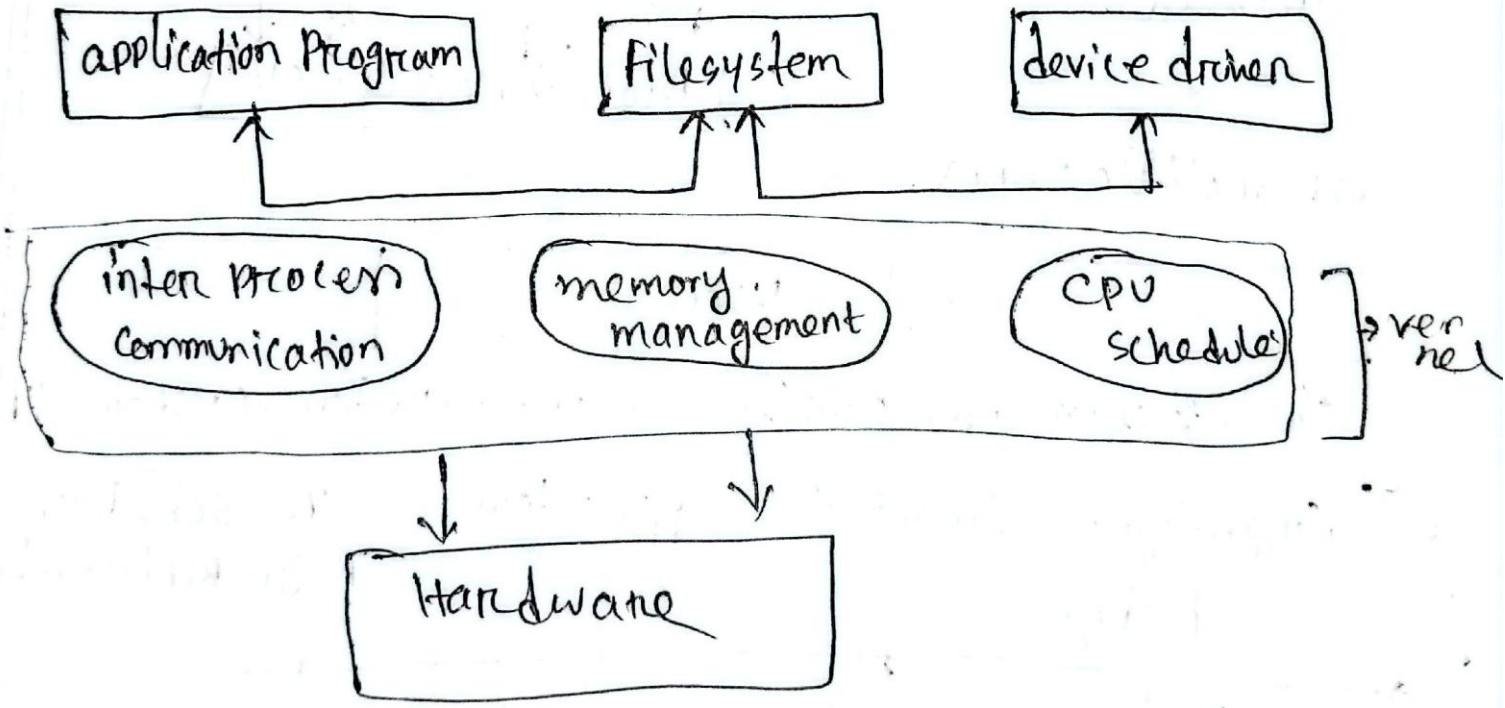


এই layer শুধু নিচের layer এ access করতে পারবে, আর অন্য layer এ পারবে না but উপরের layer গুলো পারবে,

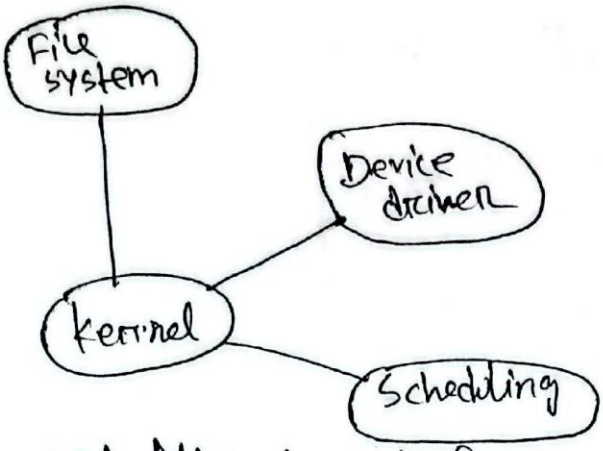
> Micro kernel:

difference

* monolithic এ অনেক কাজ এখন user mode এ হচ্ছে এইখানে kernel mode এ হবে।



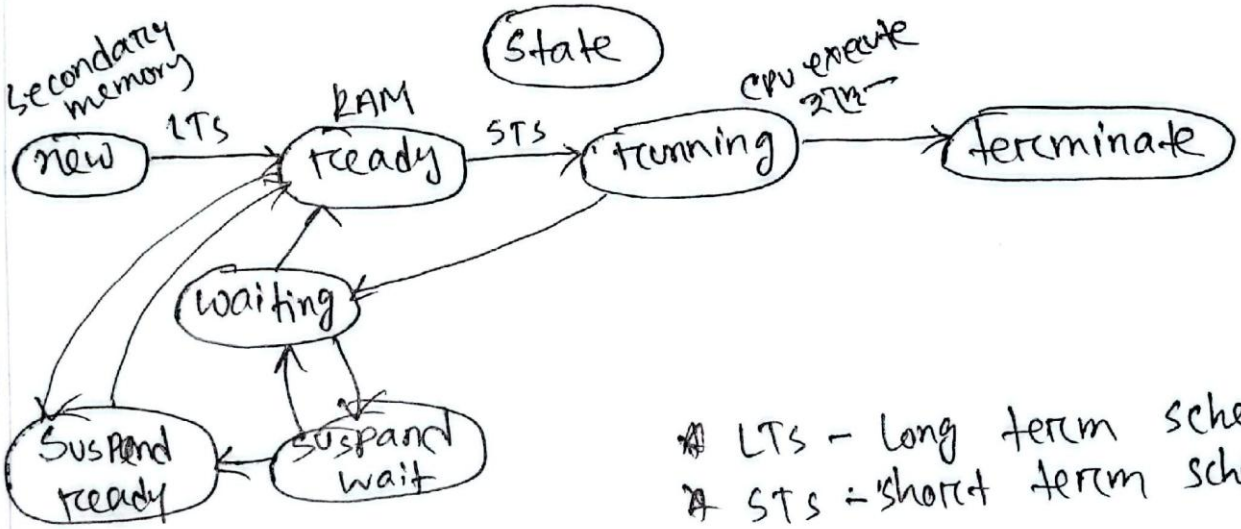
→ Module:



* Monolithic and Micro kernel মিশ্রে window structure create হয়।
* Monolithic + Module mix করে linux structure create হয়।

Process

* কোন task কে execute করাকে Process বলে।

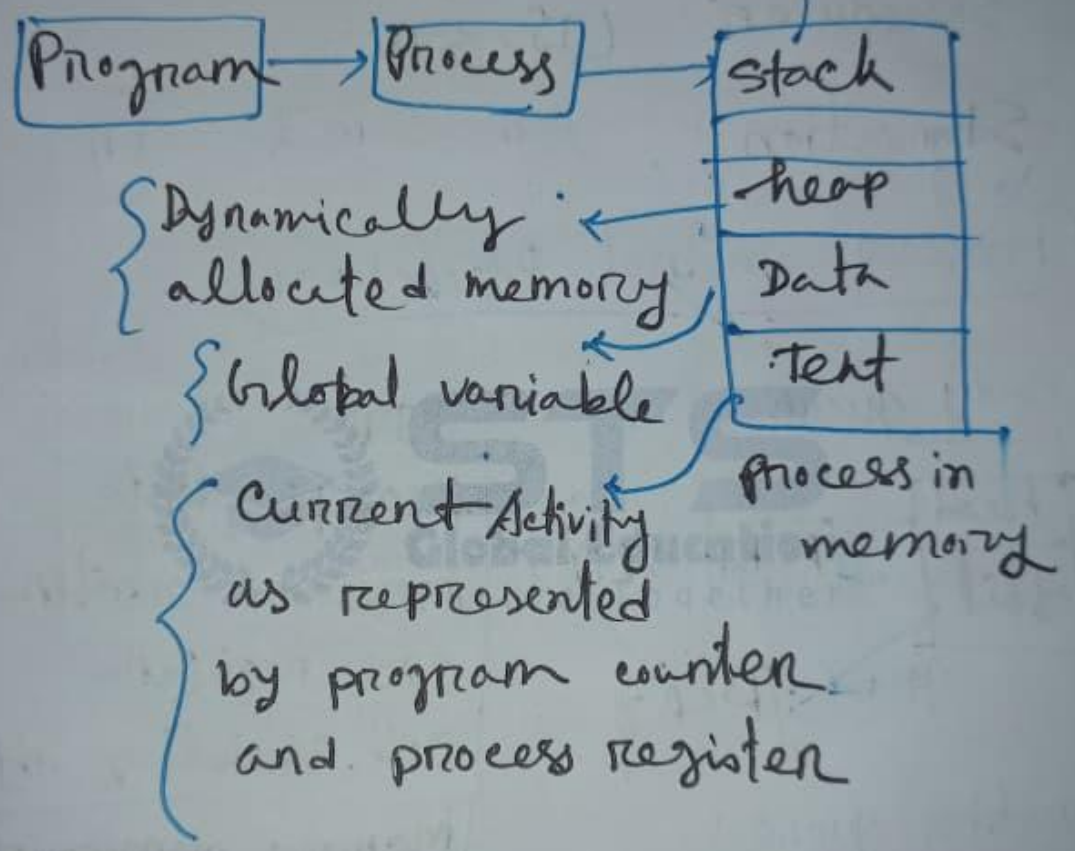


* LTS - Long term scheduler
* STS - Short term schedule

Date. 08.02.26

OS:

Process:



Process State:

new:
ready:
running:
waiting ~~ready~~:
suspend ready:
suspend ~~ready~~ wait:
terminate:

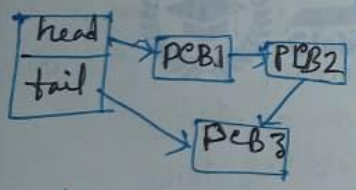
- I/O bound
- CPU bound

Scheduler: LTS, STS

Starvation:

* Process control Block (PCB)

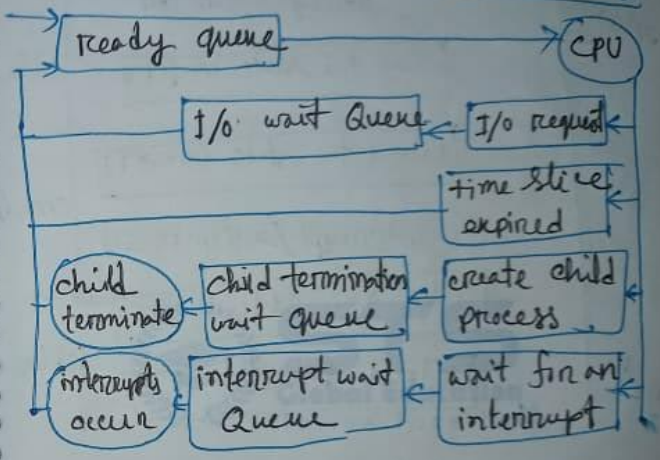
ready queue:



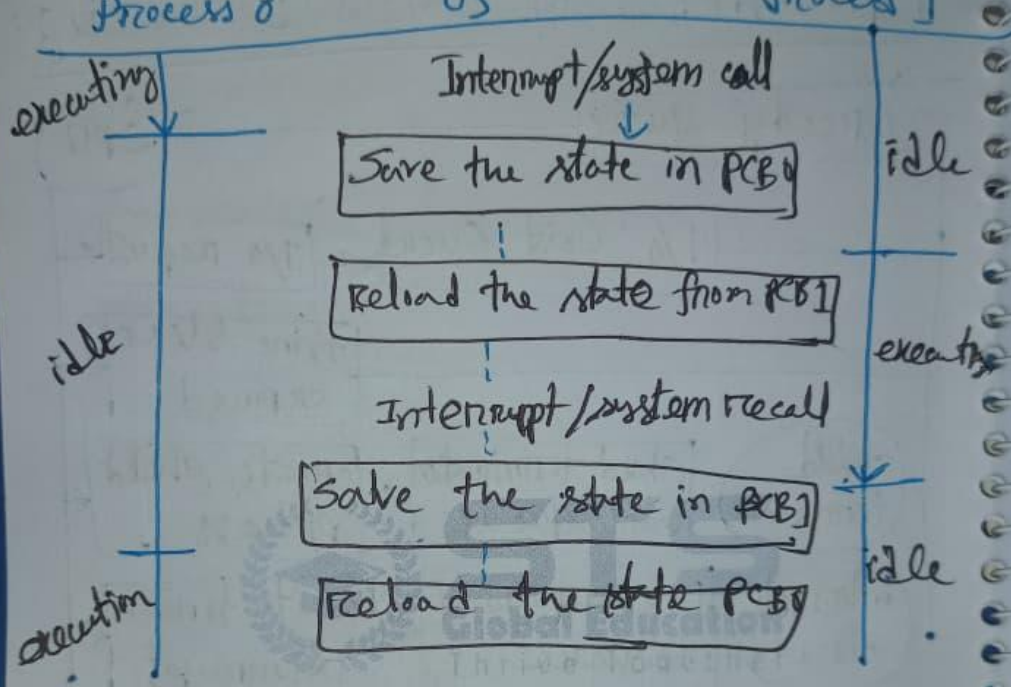
waiting queue:

PID
Process state
Program counter
CPU register
CPU scheduling info
Memory management info
I/O status info
Accounting info

Representation of Process Scheduling:



CamScanner CS



Q. Dispatcher latence ?

context switching.

* Process need separate memory
* thread works in one memory by sharing

Thread:

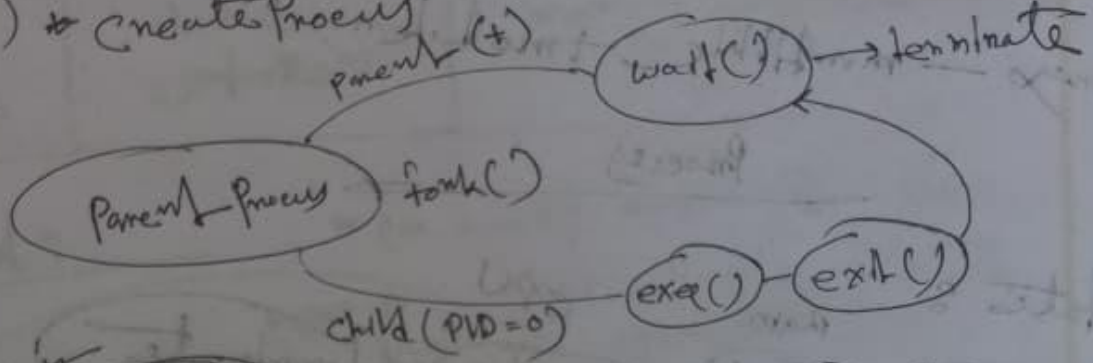
CamScanner CS

16-02-2026

Operation on Process

* Parent Process-PID → process creation
  - child Process-PID
  - child process-PID
  → process termination

* fork() * create Process
  - \- / 0 / +



* * * Unix system → * abort() ? Reasons?

* cascading termination * * Orphan * * Zombie *

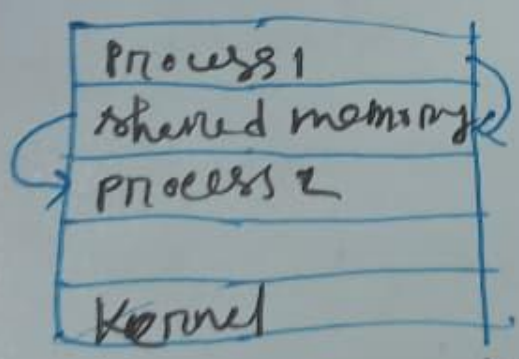
Inter process communication

- Shared memory
- Message passing

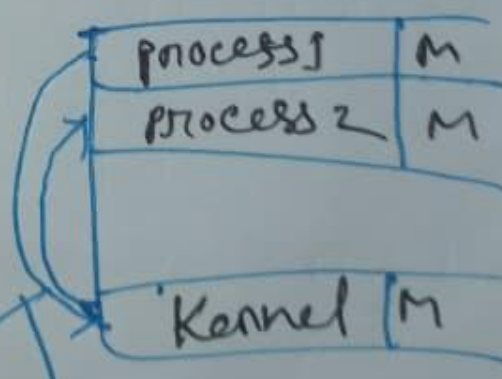
* abort() কখন call হয়ে থাকে ??
* cascading termination ??
* orphan ??
* zombie ??

Interprocess communication :

- shared memory
- message passing (distributed system)



direct



indirect

Q why need interprocess communication

OSP

Inter Process Communication:

Message Passing:

- Direct / indirect
- Synchronous / Asynchronous
- Buffer

Direct: (Name explicity)

Send (P, msg) → send a msg to P.
Receive (Q, msg) → receive a msg from Q.

asymetric in addressing

properties of communication link:-

→ Link is established automatically.
→ A link is associated with exactly one pair of communication.

→ Between each pair there exists exactly one link.
→ Link maybe undirection but is usually bidirection.

asymetric in addressing:

send (p, msg) → send a msg to p.
receive (id, msg) → receive a msg from id

Indirect: (Mail box used to send msg and receive msg with mail box id.)

→ Id
→ Shared mailbox.

Send (A, msg) → send msg to mail box A
receive (A, msg) → receive msg from mailbox A.

CamScanner CS

Properties of Link

→ ~~Shared~~ A Link may be associated with multiple process.

→ Each pair of process may be share several communication links.

→ Links ~~may~~ can be unidirectional and bidirectional.

Blocking/synchronous
Non-blocking/Asynchronous

Blocking Send → The sender is blocked untill the msg is received.

Blocking Receive → The receiver is blocked untill a msg is available.

non-blocking send → The sender sends the msg and continue.

non-blocking receive → The receiver receives a valid msg/null.

→ rendezous ?

Buffering:

- → zero capacity (only one msg)
- → Bounded capacities (finite amount msg)
- → Unbounded capacities (Non finite amount msg)

CamScanner CS

STS Global Education Thrive Together

Date. 23.02.26

OSI

* Communication in client Server System

→ Socket

→ RPC (Remote Procedure Call)

host x

19.6.86.5.20 : 1625 — 1024

webserver

161.25.19.8180

client-side (client stub) —— Server (Server-stub)

* Arrival time.
* Response time
* Waiting "
* Burst time
* Turn around time.

← CPU scheduling

STS Global Education Thrive Together

Date.....

* throughput ? per unit time output.

CPU Scheduling

- → FCFS
- → SJF
- → Round Robin
- → Priority Base

* FCFS

Process	Burst time	arrival	Waiting time
P1	3	0	0
P2	4	2	1
P3	5	3	4

P1	P2	P3	
0	3	7	12

CamScanner CS

STS Global Education — Thrive Together

Date.....

*SJF

Process	Burst time	Arrival	Waiting
P1	5	0	7
P2	3	2	0
P3	4	3	0

P2	P3	P1

0 3 7 12

*Round Robin → time quantum.

Process	Burst time	Arrival	Waiting
P1	5	0	6
P2	3	2	4
P3	4	3	4

P1	P2	P3	P1	P2	P3	P1

0 2 4 6 8 9 11 12

STS Global Education — Thrive Together

Date.....

*Priority Base:

Priority	Process	Burst	Arrival	Waiting
1	P1	5	0	6
3	P2	3	2	4
2	P3	4	3	4

P2	P3	P1

0 3 7 12

CamScanner CS